mcp-lock.json: A Committable Lock Format for MCP Client Configurations

Prachet Poddar prachetpoddar@gmail.com

Draft 1, 8 September 2026

This is a proposal, not an adopted specification. It is offered for discussion. A reference implementation and a worked example are released alongside it, and every claim about behaviour in this document is exercised by a test in that implementation.

Status

Independent work, not affiliated with any organisation and not funded.

The format is implemented, the reproducibility property is tested rather than asserted, and the design decisions that follow are supported by measurements released with the study this came out of. Section 12 lists what the format does not cover, including one question about npm's behaviour that is recorded as unsettled rather than guessed at.

1. Motivation

An MCP client configuration names servers and how to start them. In practice most entries invoke a package manager at launch:

```
{ "command": "npx", "args": ["-y", "@modelcontextprotocol/server-filesystem"] }
```

Nothing there is pinned. The version that runs is whatever the registry serves at that moment, and the dependency tree behind it is materialised at install time and recorded nowhere a reviewer can read.

Three measurements motivate the format. All are from a corpus of 250 npm-published MCP servers, released with the study cited in Section 14.

The tree is much larger than the configuration suggests. The median server installs 93 packages and the largest installs 618. What the manifest declares is a median of about 3% of that.

The tree is not flat. 68% of the 250 trees contain at least one package name at two or more versions, so a format that can hold only one version per name misdescribes two trees in three.

Across servers it is worse. On a fourteen-server configuration, 50 of 351 distinct package names resolve to different versions for different servers. A format that keeps one global map silently discards one of them.

Existing MCP pinning writes machine-local state keyed on the absolute path of the configuration file. That file is machine-specific by construction and cannot be shared even if it is committed. The thing teams want is what npm already gave them for application dependencies: a file committed next to the configuration, reviewed in a pull request, that makes every machine and every CI run materialise the same tree.

2. Terminology

The key words MUST, MUST NOT, REQUIRED, SHALL, SHALL NOT, SHOULD, SHOULD NOT, RECOMMENDED, MAY and OPTIONAL in this document are to be interpreted as described in RFC 2119 and RFC 8174, when and only when they appear in all capitals.

Lock file: the JSON document defined here, normally named `mcp-lock.json`.

Snapshot file: the companion document defined in Section 9, normally named `mcp-lock.snapshot.json`.

Generator: a program that produces a lock file from a configuration.

Consumer: a program that installs or verifies from a lock file.

Runtime: how a server is started. This document defines `npm`, `pypi`, `oci` and `remote`.

3. File placement

The lock file SHOULD sit beside the configuration it describes and SHOULD be committed. It MUST NOT contain an absolute path, a home directory, a user name, a host name or any other machine identifier. Two people who check out the same repository and generate from the same snapshot MUST produce byte-identical lock files.

4. Top-level structure

A lock file is a JSON object. It MUST contain `lockfileVersion`, `servers`, `packages` and `integrityOfLock`. It MUST contain `duplicates`, `conflicts`, `unlockable`, `resolutionFailures` and `review`, each of which MAY be empty. It SHOULD contain `resolutionSnapshot`.

Key	Type	Required	Meaning
`lockfileVersion`	integer	MUST	Format version. This document defines 1.
`servers`	object	MUST	One entry per server in the configuration. Normative.
`packages`	object	MUST	One entry per distinct package name. Informative.
`duplicates`	object	MUST	Every version of every name that has more than one. Normative.
`conflicts`	object	MUST	Which servers get which version, for each such name.

Key	Type	Required	Meaning
`unlockable`	object	MUST	Servers this file does not lock, with reasons.
`resolutionFailures`	object	MUST	Edges that did not resolve, per server.
`resolutionSnapshot`	object	SHOULD	Digest of the registry state resolved against.
`review`	object	MUST	Summary of what a human reviewer must look at.
`integrityOfLock`	string	MUST	SHA-256 over the document with this key removed.

Consumers MUST reject a lock file whose `lockfileVersion` they do not understand. They MUST NOT ignore unknown top-level keys silently; an unknown key SHOULD produce a warning naming it.

5. The `servers` object, and what is authoritative

Each key is a server name from the configuration. Each value MUST contain `runtime`.

A consumer installing a server MUST use that server's own resolved versions. For `npm` servers this is `resolvedVersions`, an object mapping package name to version. The `packages` object in Section 6 is a summary for human review and MUST NOT be used to decide what to install.

This is the central rule of the format and it exists because the alternative was implemented first and was wrong. A single global map, one version per name, silently kept whichever server was processed last. On a real fourteen-server configuration that map pinned `iconv-lite` at 0.7.3 while one server reached it through `body-parser@1.20.6`, which declares `~0.4.24`. Any consumer honouring that map installs a version violating a declared range, and a verifier comparing by name agrees with it. A lock that is confidently wrong is worse than no lock, because no lock leaves you uncertain, which is a recoverable state.

5.1 `runtime: "npm"`

Field	Type	Required	Meaning
`package`	string	MUST	Package name of the server itself.
`requested`	string	MUST	The range or tag the configuration asked for.
`version`	string	MUST	The version that

Field	Type	Required	Meaning
			range resolved to.
`integrity`	string	MUST	Subresource Integrity string for that version.
`resolvedVersions`	object	MUST	name to an array of versions, for the whole tree. Normative.
`packageCount`	integer	MUST	Size of the tree.
`command`	string	SHOULD	The command from the configuration.
`consumesEnv`	array	SHOULD	Names of environment variables the server reads.
`inTreeDuplicates`	integer	MAY	Count of names appearing at more than one version.
`unresolvedEdges`	integer	MAY	Count of edges that did not resolve.
`truncated`	boolean	MAY	Present and true if a generator cap was hit.

`resolvedVersions` MUST include every package the tree materialises, including non-optional peer dependencies, which npm 7 and later install automatically.

Each value MUST be an array of versions, even when there is one. A tree can contain a name at two versions, and npm nests both. Writing this field as name to a single version reproduces, inside the authoritative field, the exact defect this format exists to remove. It was written that way first, and seven of the nine npm servers in the reference configuration lost at least one version to it. The array is required in every case rather than a string when single and an array when not, so that a consumer never branches on the type.

`packageCount` MUST equal the total number of versions across `resolvedVersions`, not the number of distinct names. A generator SHOULD assert this, since the two diverge exactly when a tree carries a duplicate.

5.2 `runtime: "pypi"`

MUST contain `package`, `version` and `integrity`. MUST contain `transitiveUnresolved` set to true if the generator did not resolve the transitive closure. PyPI requirement metadata needs environment marker evaluation that is genuinely machine-dependent, so a generator that cannot compute the closure MUST say so rather than record a root as though it were one.

5.3 `runtime: "oci"`

MUST contain `image`. MUST contain either `digest`, a resolved manifest digest, or `digestUnresolved` set to true. A server with `digestUnresolved` MUST appear in `review.serversNotLocked`, because a tag is mutable and such a server is not locked.

A generator MUST NOT write a placeholder string into `digest`. An earlier version of the reference implementation wrote `"digest": "not-resolved"`, which is a field that looks like a lock and is not one, and which sat among the locked servers where a reader scanning for gaps would not find it.

5.4 `runtime: "remote"`

MUST contain `url`. Nothing is installed, so nothing is pinned. The endpoint is recorded so that a change to it is visible in a diff.

6. The `packages` object

Keys are package names. Each value MUST contain `version`, `integrity` and `servers`, the list of servers that get that version. It MAY contain `resolved`, `optional`, `os`, `cpu` and `installScripts`.

Where a name has more than one version, the entry here MUST be the version used by the largest number of servers, with ties broken toward the higher version, so that the choice is deterministic across runs and machines and so that a reader skimming the body sees the typical case rather than an artifact of sort order.

This object is informative. It exists because a name-keyed body produces a readable diff, and readability is a security property: `package-lock.json` is path-keyed and famously unreadable, and that unreadability is why lockfile poisoning works, because a malicious version bump hides in a twenty-thousand-line diff nobody reads.

7. The `duplicates` object

Keys are package names that resolve to more than one version. Each value maps version to a record carrying **the same fields as a `packages` entry**.

Duplicates MUST NOT be an appendix. At roughly one tree in three, and one name in seven across a multi-server configuration, duplicates are a normal part of the file rather than an exception, and a list tucked below the body is a list nobody reads. Carrying reduced fields here would mean a scanner reading body plus duplicates loses information that a scanner reading a path-keyed lock would have, which would give up the only thing path-keying is better at.

8. The `conflicts` object

Keys are the same names as `duplicates`. Each value MUST contain:

Field	Type	Meaning
`versions`	array	Every version, sorted.
`byServer`	object	version to the list of servers receiving it.

Field	Type	Meaning
`inBody`	string	Which version Section 6 chose.

A generator MUST write this object whenever a name resolves to more than one version. It MUST NOT omit the object when the condition holds.

That requirement is stated because the reference implementation violated it. It detected the collision, created an empty set, never added to it, and the serializer removed the key before writing, so the mechanism built to declare conflicts reported success by being absent. Nothing failed. The file was clean and wrong.

9. The `resolutionSnapshot` object and the snapshot file

`resolutionSnapshot` MUST contain `file`, `sha256`, `packages` and `capturedUtc`.

Integrity hashes pin what you get. They do not pin the shape of the tree, because a newly published transitive version changes the set of packages installed without changing the hash of any package already in the file. Without an anchor, the same configuration produces a different lock on different days and nothing in the file says why, which also makes registry movement indistinguishable from a configuration change during verification.

The snapshot file MUST contain `capturedUtc` and `packages`. `packages` maps package name to:

Field	Type	Meaning
`all`	array	Every version that existed at capture time.
`deprecated`	array	Those marked deprecated. A subset of `all`.
`distTags`	object	The dist-tag map at capture time.
`used`	object	Full metadata for versions actually placed. Keys MUST be a subset of `all`.

`used` carries only what resolution and the lock record need: dependency groups, `peerDependenciesMeta`, `os`, `cpu`, install-time scripts, and `dist`. Storing a per-version object for every version cost 1.1 MB on a fourteen-server configuration, most of it empty objects for packages such as `playwright` with 5,676 published versions.

A consumer replaying from a snapshot MUST fail loudly when the snapshot cannot answer a resolution the original made. It MUST NOT record such a case as an ordinary missing package, because a replay that quietly resolves a smaller tree would be the silent degradation this format exists to remove. A snapshot whose `used` is not a subset of `all` MUST be rejected as corrupt.

Conformance requirement. A generator claiming this format MUST be able to regenerate a byte-identical lock file from its own snapshot with no network access. This is the only property in this document that cannot be checked by inspecting a file, and it is the one most worth testing.

10. `unlockable`, `resolutionFailures` and `review`

`unlockable` maps server name to `spec` and `reason`. Local paths, mutable Docker tags and git specifiers cannot be pinned from a registry. They MUST be listed so that the file never implies more coverage than it has.

`resolutionFailures` maps server name to the edges that did not resolve. A generator MUST record every discarded edge somewhere that reaches the file. A count printed to a terminal is not a record.

`review` is a summary of what a human must look at. It MUST contain `serversNotLocked`, `installScripts` and `serversConsumingSecrets`. It SHOULD contain `namesAtMoreThanOneVersion`, `conflictingNames`, `packagesWithoutIntegrity`, `platformConstrained`, `remoteEndpoints`, `serversPinnedRootOnly` and `serversWithResolutionFailures`.

Install scripts belong at the top of the file because they are the set a reviewer must actually examine. `package-lock.json` records `hasInstallScript` and buries it. In the study corpus 15.6% of trees run install-time code, so this is not a rare case.

11. Credentials

A generator MUST NOT read or write the value of any environment variable named in a configuration. It MUST record only the variable names, under `consumesEnv`, so that a reviewer sees a new secret being introduced in a diff.

MCP configurations routinely carry live API keys in their `env` blocks. A lock file is committed. This requirement MUST be enforced in code and MUST be covered by a test that fails if any configured value appears anywhere in the output.

12. What this format does not cover

Stated rather than left to discovery.

OCI manifest digests. Resolving a tag to a digest needs a registry token. Servers whose digest is unresolved are declared unlocked.

PyPI transitive closures. Environment markers make the closure machine-dependent. Roots are pinned; the closure is not.

Generator caps. The reference implementation stops at 2,000 packages per server and sets `truncated`. A truncated tree is declared, not silently smaller.

One npm behaviour is unsettled. npm prefers the version tagged `latest` when it satisfies a range, and prefers a non-deprecated version over a deprecated one. What it does when the `latest` tag is itself deprecated and satisfies is not established here. The reference resolver takes the tag without consulting the deprecation flag, and every case validated against a real install was a range where `latest` did not satisfy, so that path has never been exercised. Mutation testing surfaced it: marking a selected version deprecated left the output unchanged. Settling it needs a real install against a

package in that state. It is recorded rather than guessed at, because guessing is what produced the four resolver defects described in Section 13.

13. Security considerations

A generator that computes a dependency tree has reimplemented its package manager's version selection, and MUST be validated against a real install rather than against its own logic. The reference resolver had four defects, each silent, each changing a published number, and all four fell out of the first comparison with a real `npm install`: npm prefers the `latest` dist-tag when it satisfies a range even if a higher version exists; npm avoids deprecated versions when a non-deprecated one satisfies; npm 7 and later install non-optional peer dependencies automatically; and npm reuses an already-placed satisfying version rather than resolving each edge independently. The first biases toward newer versions than the machine has, so a scanner reading such a tree under-reports vulnerabilities. The last inflates the tree and over-reports.

A lock file is not a vulnerability report. This format carries no advisory or severity data. What will be installed is a fact that changes when the configuration changes. Whether it is safe is a question for a scanner reading the lock, and the answer changes daily. Mixing them makes the diff churn and the file stop being reviewable.

Staleness is the failure mode nobody plans for. A validly signed document with no lifetime replays forever and nothing notices. This is the argument behind TUF's timestamp role, and it applies to any artifact describing a dependency state at a point in time. In the study corpus, comparing a tree frozen at release against the tree installed today changed the advisory verdict for 64.8% of servers, almost entirely by accumulating findings the running system does not have. `capturedUtc` exists so that a consumer can tell how old the description is.

14. Reference implementation and conformance

`mcp_lock.py` generates, replays and verifies. `mcp-lock.json` and `mcp-lock.snapshot.json` are a worked example over a fourteen-server configuration.

A conforming implementation SHOULD be checked against at least these, each of which the reference implementation is tested against by planting the defect and requiring a complaint:

1. Regenerating from its own snapshot with no network produces a byte-identical lock file.
2. Removing a package from the snapshot causes a loud failure, not a smaller tree.
3. Removing the selected version from a snapshot's `all` changes the resolved version.
4. A `used` version absent from `all` is rejected as corrupt.
5. Deleting a conflict from a lock file makes verification fail and name it.
6. Changing one server's entry in `resolvedVersions`, with the body untouched, makes verification fail and name it.
7. `packageCount` equals the total number of versions in `resolvedVersions` for every server, which fails if the map has collapsed a duplicate.
8. No configured environment variable value appears anywhere in the output.

Test 6 exists because verification originally compared only the body, and so agreed with a lock that had silently overwritten one server's version with another's.

Study, data and code: https://github.com/prachetpoddar/mcp-supply-chain-study
Archived at DOI 10.5281/zenodo.22664719.

15. Acknowledgements

Adam Dudley of `mcp-audit` argued that optimising for reviewability is the security choice rather than the ergonomic one, and that at these frequencies the duplicate list has to be a first-class section rather than an overflow bucket. Both shaped Sections 6 and 7. The observation that a signed document without a lifetime replays forever is TUF's, by way of his advisory feed.